

Exercises

23.3-1

Use Johnson's algorithm to find the shortest paths between all pairs of vertices in the graph of Figure 23.2. Show the values of h and $\hat{w}$ computed by the algorithm.

23.3-2

What is the purpose of adding the new vertex s to V , yielding V' ?

23.3-3

Suppose that $w(u, v) \geq 0$ for all edges $(u, v) \in E$. What is the relationship between the weight functions w and $\hat{w}$?

23.3-4

Professor Greenstreet claims that there is a simpler way to reweight edges than the method used in Johnson's algorithm. Letting $w^* = \min \{w(u, v) : (u, v) \in E\}$, just define $\hat{w}(u, v) = w(u, v) - w^*$ for all edges $(u, v) \in E$. What is wrong with the professor's method of reweighting?

23.3-5

Show that if G contains a 0-weight cycle c , then $\hat{w}(u, v) = 0$ for every edge (u, v) in c .

23.3-6

Professor Michener claims that there is no need to create a new source vertex in line 1 of JOHNSON. He suggests using $G' = G$ instead and letting s be any vertex. Give an example of a weighted, directed graph G for which incorporating the professor's idea into JOHNSON causes incorrect answers. Assume that $\infty - \infty$ is undefined, and in particular, it is not 0. Then show that if G is strongly connected (every vertex is reachable from every other vertex), the results returned by JOHNSON with the professor's modification are correct.

Problems

23-1 Transitive closure of a dynamic graph

You wish to maintain the transitive closure of a directed graph $G = (V, E)$ as you insert edges into E . That is, after inserting an edge, you update the transitive closure of the edges inserted so far. Start with G having no edges initially, and represent the transitive closure by a boolean matrix.

- a. Show how to update the transitive closure $G^* = (V, E^*)$ of a graph $G = (V, E)$ in $O(V^2)$ time when a new edge is added to G .
- b. Give an example of a graph G and an edge e such that $\Omega(V^2)$ time is required to update the transitive closure after inserting e into G , no matter what algorithm is used.
- c. Give an algorithm for updating the transitive closure as edges are inserted into the graph. For any sequence of r insertions, your algorithm should run in time $\sum_{i=1}^r t_i = O(V^3)$, where t_i is the time to update the transitive closure upon inserting the i th edge. Prove that your algorithm attains this time bound.

23-2 Shortest paths in ϵ -dense graphs

A graph $G = (V, E)$ is **ϵ -dense** if $|E| = \Theta(V^{1+\epsilon})$ for some constant in the range $0 < \epsilon \leq 1$. d -ary min-heaps (see Problem 6-2 on page 179) provide a way to match the running times of Fibonacci-heap-based shortest-path algorithms on ϵ -dense graphs without using as complicated a data structure.

- a. What are the asymptotic running times for the operations INSERT, EXTRACT-MIN, and DECREASE-KEY, as a function of d and the number n of elements in a d -ary min-heap? What are these running times if you choose $d = \Theta(n^\alpha)$ for some constant $0 < \alpha \leq 1$? Compare these running times to the amortized costs of these operations for a Fibonacci heap.
- b. Show how to compute shortest paths from a single source on an ϵ -dense directed graph $G = (V, E)$ with no negative-weight edges in

$O(E)$ time. (*Hint: Pick d as a function of ϵ .*)

- c. Show how to solve the all-pairs shortest-paths problem on an ϵ -dense directed graph $G = (V, E)$ with no negative-weight edges in $O(VE)$ time.
- d. Show how to solve the all-pairs shortest-paths problem in $O(VE)$ time on an ϵ -dense directed graph $G = (V, E)$ that may have negative-weight edges but has no negative-weight cycles.

Chapter notes

Lawler [276] has a good discussion of the all-pairs shortest-paths problem. He attributes the matrix-multiplication algorithm to the folklore. The Floyd-Warshall algorithm is due to Floyd [144], who based it on a theorem of Warshall [450] that describes how to compute the transitive closure of boolean matrices. Johnson's algorithm is taken from [238].

Several researchers have given improved algorithms for computing shortest paths via matrix multiplication. Fredman [153] shows how to solve the all-pairs shortest paths problem using $O(V^{5/2})$ comparisons between sums of edge weights and obtains an algorithm that runs in $O(V^3 \lg \lg V / \lg V^{1/3})$ time, which is slightly better than the running time of the Floyd-Warshall algorithm. This bound has been improved several times, and the fastest algorithm is now by Williams [457], with a running time of $O(V^3 / 2^{\Omega(\lg^{1/2} V)})$.

Another line of research demonstrates how to apply algorithms for fast matrix multiplication (see the chapter notes for Chapter 4) to the all-pairs shortest paths problem. Let $O(n^\omega)$ be the running time of the fastest algorithm for multiplying two $n \times n$ matrices. Galil and Margalit [170, 171] and Seidel [403] designed algorithms that solve the all-pairs shortest paths problem in undirected, unweighted graphs in $(V^\omega p(V))$ time, where $p(n)$ denotes a particular function that is polylogarithmically bounded in n . In dense graphs, these algorithms are faster than the $O(VE)$ time needed to perform $|V|$ breadth-first

searches. Several researchers have extended these results to give algorithms for solving the all-pairs shortest paths problem in undirected graphs in which the edge weights are integers in the range $\{1, 2, \dots, W\}$. The asymptotically fastest such algorithm, by Shoshan and Zwick [410], runs in $O(W^{1/\omega} p(V, W))$ time. In directed graphs, the best algorithm to date is due to Zwick [467] and runs in $\tilde{O}(W^{1/(4-\omega)} V^{2+1/(4-\omega)})$ time.

Karger, Koller, and Phillips [244] and independently McGeoch [320] have given a time bound that depends on E^* , the set of edges in E that participate in some shortest path. Given a graph with nonnegative edge weights, their algorithms run in $O(VE^* + V^2 \lg V)$ time and improve upon running Dijkstra's algorithm $|V|$ times when $|E^*| = o(E)$. Pettie [355] uses an approach based on component hierarchies to achieve a running time of $O(VE + V^2 \lg \lg V)$, and the same running time is also achieved by Hagerup [205].

Baswana, Hariharan, and Sen [37] examined decremental algorithms, which allow a sequence of intermixed edge deletions and queries, for maintaining all-pairs shortest paths and transitive-closure information. When a path exists, their randomized transitive-closure algorithm can fail to report it with probability $1/n^c$ for an arbitrary $c > 0$. The query times are $O(1)$ with high probability. For transitive closure, the amortized time for each update is $O(V^{4/3} \lg^{1/3} V)$. By comparison, Problem 23-1, in which edges are inserted, asks for an incremental algorithm. For all-pairs shortest paths, the update times depend on the queries. For queries just giving the shortest-path weights, the amortized time per update is $O(V^3/E \lg^2 V)$. To report the actual shortest path, the amortized update time is $\min \{O(V^{3/2} \sqrt{\lg V}), O(V^3/E \lg^2 V)\}$. Demetrescu and Italiano [111] showed how to handle update and query operations when edges are both inserted and deleted, as long as the range of edge weights is bounded.

Aho, Hopcroft, and Ullman [5] defined an algebraic structure known as a “closed semiring,” which serves as a general framework for solving path problems in directed graphs. Both the Floyd-Warshall algorithm

and the transitive-closure algorithm from Section 23.2 are instantiations of an all-pairs algorithm based on closed semirings. Maggs and Plotkin [309] showed how to find minimum spanning trees using a closed semiring.

¹ According to a report cited by U.S. Department of Transportation Federal Highway Administration, “a reasonable ‘rule of thumb’ is one signalized intersection per 1,000 population.”

² An algebraic *semiring* contains operations $\oplus$, which is commutative with identity $I_\oplus$, and $\otimes$, with identity $I_\otimes$, where $\otimes$ distributes over $\oplus$ on both the left and right, and where $I_\otimes \otimes x = x \otimes I_\otimes = I_\otimes$ for all x . Standard matrix multiplication, as in MATRIX-MULTIPLY, uses the semiring with $+$ for $\oplus$, $\otimes$ for $\otimes$, 0 for $I_\oplus$, and 1 for $I_\otimes$. The procedure EXTEND-SHORTEST-PATHS uses another semiring, known as the *tropical semiring*, with min for $\oplus$, $+$ for $\otimes$, ∞ for $I_\oplus$, and 0 for $I_\otimes$.